EB PEARLS · QA

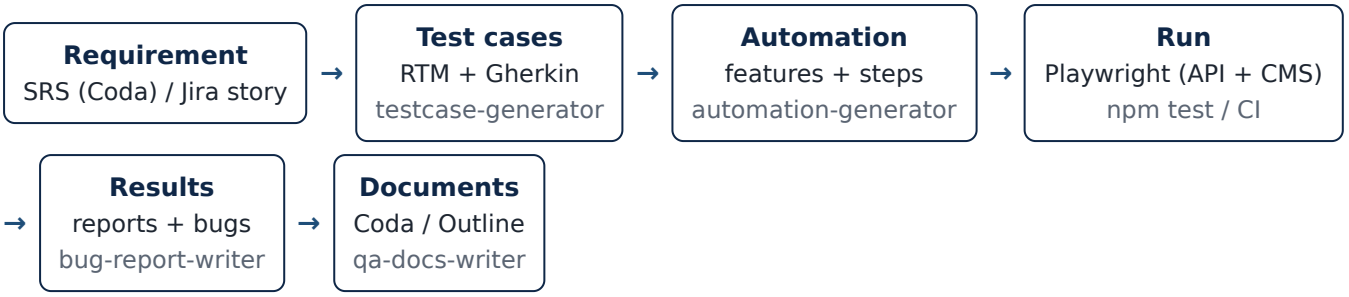
SPG QA Package
How Everything Works

Bug reports, the verified Postman collection, the Gherkin + Playwright automation for the API and CMS, and the AI QA skills – explained in plain language.

Environment: SPG staging (stagecms / stageapi.red.global) · Prepared 7 October 2026
Built to the EB Pearls QA Process Guidelines · Works on Linux · Skills for OpenClaude, Claude Code and Opencode

1. The big picture

Everything in this package follows one loop. A requirement becomes test cases, the test cases become automated Gherkin scenarios, the scenarios run against SPG, and the results become bug reports and sprint documents.



What is in the package

Folder / file	What it is	Who uses it
`START_HERE.md`	One-page map of the package	Everyone
`SPG_API_QA_Report_…/` Word files	Confirmed bugs (short + detailed), suspected bugs and how to verify	Senior, EM, developers
`…/markdown/`	Same reports as Markdown + bug tracker CSV	Coda, Outline, Zoom Docs
`…/postman/`	Verified Postman collection v2.1 (115 requests, creates no permanent records) + “Creates Permanent Data” collection (39 requests, on purpose only) + environment + upload photo	Manual / API QA
`SPG_Jira_Bug_Reporting_2026-10-08/`	How to work in Jira (time log, bug tickets, linking to stories) + 10 bug tickets ready to paste + cleanup request for devs	Daily Jira work
`spg-qa-automation/`	Gherkin + Playwright project, CI pipeline, AI skills, guides	Automation QA

> **Passwords are not included anywhere.** You set them yourself: Postman *Current value* and the `.env` file.

2. How SPG works (what a tester must know)

CMS and API

The CMS (stagecms) is a React app. Every screen calls the SPG API (stageapi …/v1). Testing the API tests the same rules the CMS uses.

One session per account

A new login ends the previous one. If Postman, the CMS and a teammate share an account, they log each other out with **"Invalid Token!"**. The automation logs in once and shares that session.

How photos work

The CMS uploads the picture straight to cloud storage (S3, via a Cognito identity), then saves only the file **key** in fields like `profilePhoto` or `entityLogo`. The API never receives the image itself.

Entity context

Entity screens send the header `x-effective-role: entityPrincipalAdmin` – the same as "Login on behalf". Client, alert-group and emergency calls need it.

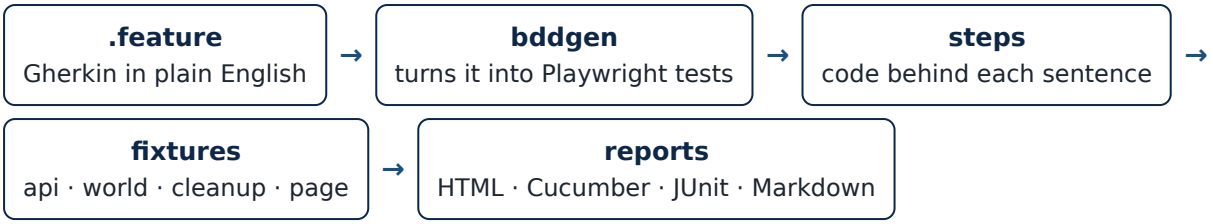
Updates need the full record

PATCH requests must send the whole object (same shape as create). Partial bodies are rejected – and on "my profile" they **erase** fields (SPG-BUG-11).

Emergencies

Start = `POST /admin/emergency`. Mark as safe = `PATCH /admin/emergency/{id}` with `{}` – it works. **Posting a response comment crashes staging** (SPG-BUG-24) – never automate it.

3. The automation – how a test runs




```
@ui @photo
Feature: Photos are shown in the CMS
  @regression @fail @SPG-BUG-25 @TC-UI-002
  Scenario: Super admin photo is shown after saving
    Given a super admin exists
    When I open the super admin edit page
    And I upload and crop the photo "qa-photo.png"
    And I save the form
    And I reopen the edit page
    Then the saved photo is shown
```

Building blocks

Piece	Job
`api`	Calls the SPG API with the shared session; logs in again if another login ended it.
`world`	Remembers IDs and request bodies between the steps of one scenario.
`cleanup`	Deletes everything the scenario created – even when it fails. QA entities are left disabled (no delete endpoint).
CMS sign-in	UI tests put the API token into the browser (`serviceToken`) instead of typing the password – a second login would log out the API tests.
`redact()`	Removes passwords, hashes and tokens from every failure message and report.

Tags (QA guide)

Tag	Meaning
`@smoke` / `@regression`	Quick check on every change / full pack nightly and before release.
`@wip` / `@manual`	Not ready / cannot be automated – never run by default.
`@TC-AREA-001`	The RTM test case id – links the scenario to Jira RTM.
`@fail @SPG-BUG-xx`	Known bug. The run stays green while the bug exists and goes **red when it is fixed** – then remove `@fail` .

Coverage today – 32 scenarios (27 by default + 5 `@creates-data` , run on purpose only)

Area	Scenarios
API	Login & security, support staff (create, photo, validation, paging), entities (logo + admin photo, enable/disable, principal admin, delete), clients (all sections, family photos, enable/disable, alert history), security numbers, lists & dropdowns, emergencies (start, mark as safe, delete, archive, timing)
CMS (browser)	Login error, main menu, entity page shows the right principal admin, photo shown after saving (Field Operative and Super Admin)

> **Every CMS endpoint is accounted for:** 116 endpoints – 92 tested, 18 not run on purpose (they send real messages, change shared settings or crash staging), 6 need data first, 0 forgotten. Full table: `docs/API_COVERAGE.md` .

Last verified 8 Oct 2026 (27 default scenarios): 21 pass · 5 known bugs flagged · 1 unexpected failure – TC-EN-001 failed once on timing (SPG-SUS-13) and passed 3 of 3 re-runs. The run added 0 entities, field operatives, support staff or entity users. The emergency start time is recorded against the 3 s SRS limit (2.9–3.5 s across runs).

4. Run it on Linux

One command for everything

```
./run.sh setup        # once: Node packages + Chromium (no sudo needed)
./run.sh smoke        # 5 quick scenarios
./run.sh test         # everything — API + CMS (~5 min) + sprint report
./run.sh postman      # Postman collection with newman (~2 min)
./run.sh report       # open the HTML report
./run.sh skills       # install the AI skills for Claude Code / OpenClaude
```

Run it from the package folder (where `run.sh` is). In your private copy the password is already in `spg-qa-automation/.env`.

Or directly inside the project

Every day

Command	What it does
`npm run test:smoke`	5 quick scenarios
`npm test`	Everything (about 5 minutes)
`npm run test:api / test:ui`	Only API / only CMS
`TAGS='@photo' npm test`	Any tag combination
`npm run test:headed`	Watch the browser
`npm run report`	HTML report with screenshots, video and trace
`node scripts/sprint-report.mjs "Sprint 7"`	Sprint automation report (Markdown for Coda / Outline)
`npm run steps:list`	All existing steps – reuse them

CI (GitHub Actions)

Pull requests run `@smoke`, a nightly job runs `@regression`, results go to Slack and reports are kept for 30 days. Add the password as a repository **secret** (`SPG_ADMIN_PASSWORD`) and the URLs as variables. GitLab / Bitbucket: run the same four commands.

Postman (manual / API QA)

1. Import the collection and environment from `SPG_API_QA_Report_…/postman/`.
2. Set `adminPassword` as **Current value**; set Postman's working directory to the `postman/` folder (photo upload).
3. Run the collection in order. Folders 00–13, 97, 99 should be green; folder 98 (known bugs) is red until fixed.

5. AI skills – OpenClaude, Claude Code, Opencode

Five skills follow the QA guide's "QA Skills Setup" format (front-matter + What I do, When to use me, Instructions, Trigger, Output, Format, Coverage). They live in `.opencode/skills/` (QA guide location) with a copy in `.claude/skills/`.

Skill	Give it	You get
`testcase-generator`	Story, SRS section, acceptance criteria	RTM table + Gherkin + coverage summary + open questions
`api-testcase-generator`	Endpoints / dev handoff comment	API cases, DDT CSV, Postman tests
`automation-generator`	Approved Gherkin	Feature + steps in this repo, verified by a real run
`bug-report-writer`	Failures, notes, screenshots	EB Pearls-format bugs; confirmed vs suspected
`qa-docs-writer`	Sprint data, reports	Release report, test plan, automation report (short)

Install the AI tool (Ubuntu)

```
npm install -g opencode-ai          # Opencode — QA guide standard
opencode auth login                 # connect Claude / OpenAI / Gemini …
cd spg-qa-automation && opencode    # skills load automatically
```

Checked: Opencode 1.18.35 loads all 5 skills from `.opencode/skills/`. OpenClaude and Claude Code steps: `docs/AI_TOOLS_SETUP.md`.

Use with OpenClaude or Claude Code

```
cd spg-qa-automation            # project skills in .claude/skills load automatically
# or install for every project:
bash scripts/install-skills.sh claude
```

Or simply `./run.sh skills`. Then ask in plain words, or call a skill by name, e.g. `/testcase-generator`. `CLAUDE.md` gives the AI the project rules.

Use with Opencode

Open the repo – `.opencode/skills/` and `AGENTS.md` load automatically. Global: `bash scripts/install-skills.sh opencode-global`.

> **After editing a skill** run `bash scripts/sync-skills.sh` to copy it to `.claude/skills`, then commit.

AI rules for the team

1. AI drafts, people approve (QA Lead review stays).
2. Verify everything: `bddgen`, type check and a real run before merging.
3. Confirmed vs suspected – never log a suspicion as a bug.
4. No secrets in prompts or output.

5. Never let AI run actions that harm shared environments.
6. Security findings stay high-level and on restricted pages.

6. Importing the reports

Where	How
Coda (recommended – SRS is there)	Open the `.md` file, copy all, paste into a page (Coda formats it). Tracker: type `/import` → CSV → `SPG_Bug_Tracker_…csv`. Attach the detailed Word files. Put security bugs on a restricted page.
Outline	Collection ⋯ → Import → the `.md` or `.docx`. A shared collection is visible to everyone with access; a private one only to you.
Zoom Docs	Docs → More → Import → Import local files. Markdown also via the Zoom Docs API (≤ 100 KB).
Jira RTM	Test cases per EPIC folder; the `@TC-…` tag is the RTM id; create bugs from the test execution and link them to the task.

Full step-by-step: `spg-qa-automation/docs/IMPORT_GUIDE.md`

7. Findings at a glance

Bug		In one line
SPG-BUG-01	Critical	Password hashes and push tokens are returned by the API.
SPG-BUG-11	Critical	Updating "my profile" with some fields erases the email – account locked out.
SPG-BUG-24	Critical	Posting an emergency response comment crashes the API for 5–30 s.
SPG-BUG-28	Critical	Photo storage can be listed and written without logging in.
SPG-BUG-25	High	Photos save but most CMS edit pages don't show them.
SPG-BUG-02	High	Entity page shows and edits the wrong person as Principal Admin.

28 confirmed (4 Critical, 6 High, 10 Medium, 8 Low) and 14 suspected – see the Word reports.

8. Mobile apps – how to automate them with AI

Plan for the SPG client app ("lite"/"secure") and field operative app. Not run yet – it needs the app builds and devices.

Need	Tool
Test language	Same Gherkin as web and API
Mobile driver	Appium 2 with WebdriverIO + Cucumber (in the QA guide)
Quick build check	Maestro YAML flows (optional)
Devices	LambdaTest real devices + 1–2 physical phones
AI control	Appium MCP / Maestro MCP – the AI sees the screen, taps, suggests locators

AI does the work, people approve

Step	AI	Person
Design	Writes test cases + Gherkin	QA Lead approves
Explore	Walks the app, suggests locators	QA corrects
Automate	Writes step code	Code review
Triage	Bug vs flaky vs changed screen; drafts the bug	QA confirms before Jira
Fix tests	Proposes the fix	QA accepts – no silent self-healing

Kept partly manual

Emergency audio/video quality, push notifications, real GPS walk test and the camera are checked by hand on a physical device each release. OTP uses the stage bypass code; data is set up through the API.

Full plan with rollout over 6 sprints: `docs/MOBILE_AUTOMATION_PLAN.md`.

9. Troubleshooting

You see	Why	Do this
`./run.sh` : permission denied	Unzip tool dropped the "executable" flag	`bash run.sh …` or `chmod +x run.sh`
401 "Invalid Token!"	Someone logged in with the same account	Use a dedicated QA account; the suite logs in again automatically
Chromium won't start on Linux	Missing system libraries	Set `PW_CHANNEL=chrome` in `.env` (uses installed Google Chrome) or ask IT to run `npx playwright install-deps chromium`
`bddgen` : missing step	A Gherkin line has no step code	`npm run steps:list` , reuse wording or add the step
"Expected to fail, but passed"	A known bug looks fixed	Re-check manually, then remove `@fail`
502 Bad Gateway everywhere	Staging API restarted (often SPG-BUG-24)	Wait ~30 s; check nobody posted an emergency comment
Photo saved as empty	Update clicked before the upload finished (SPG-SUS-12)	Wait for the upload spinner to finish
Missing `SPG_…` setting	`.env` not filled in	Copy `.env.example` to `.env`

Sprint checklist (QA guide)

1. Sprint planning: generate and review test cases for the next two sprints (RTM + Gherkin).
2. During the sprint: automate approved scenarios; tag `@wip` until the feature is deployed.
3. Every merge: CI runs `@smoke` ; a failing pipeline blocks the merge.
4. Regression sprint: `npm run test:regression` + Postman collection.
5. Release: sprint automation report + release report (qa-docs-writer), bugs linked in Jira.
6. Retro: review known bugs that turned green and flaky timing results.